

AI Usage Documentation

Dormitory and Cafeteria Management System (C# & SQL)

Purpose of This Document

This document formally describes how Artificial Intelligence (AI) tools were used during the development of the **Dormitory and Cafeteria Management System** projects (C# Console Application and SQL Database Project).

The purpose is to demonstrate **ethical, professional, and transparent AI usage**, fully aligned with the provided **AI Usage Policy**. AI was used strictly as a **support tool** to assist learning, debugging, and optimization — **not** as a replacement for understanding, design decisions, or full project generation.

All architectural decisions, final implementations, and validations were made by the student.

AI Usage Principles Followed

The following rules were strictly respected throughout the project:

Allowed and Encouraged Usage

- Explaining technical concepts
- Debugging specific error messages
- Suggesting improvements and best practices
- Reviewing logic and structure
- Helping reason about design alternatives

Unacceptable Usage (Explicitly Avoided)

- Copying an entire project without understanding
- Using code that cannot be explained line-by-line
- Letting AI decide system architecture
- Skipping the design or planning phase

Each AI interaction was documented with:

- What was asked
 - What AI suggested
 - What was chosen and **why**
-

AI Interaction Log

Prompt 1: Project Architecture Planning

What We Asked

We asked AI for feedback on whether separating the system into two projects (C# Console Application and SQL Database) with the same domain theme (Dormitory and Cafeteria) was architecturally correct and academically acceptable.

What AI Suggested

AI confirmed that separating application logic from data storage is a correct and realistic approach. It suggested keeping responsibilities clearly divided and using consistent naming between classes and database tables.

What We Chose and Why

We chose to keep the projects separated initially to ensure clarity, modularity, and easier debugging. This decision allowed us to fully understand each layer independently before considering integration.

Prompt 2: Database Design and Table Relationships

What We Asked

We asked AI to explain how entities such as Students, Dormitory Applications, Cafeteria Orders, and Payments should be logically related in a relational database.

What AI Suggested

AI suggested using primary and foreign keys, avoiding redundant data, and following normalization principles. It explained one-to-many relationships and real-world modeling examples.

What We Chose and Why

We designed the tables manually based on the explanation, ensuring that each table had a clear responsibility and meaningful relationships. We avoided auto-generated schemas to fully understand the design.

Prompt 3: Debugging SQL Errors

What We Asked

We asked AI to explain specific SQL errors (e.g., missing tables, relation does not exist, incorrect column names) that occurred during query execution.

What AI Suggested

AI explained the meaning of each error message, possible causes, and how to verify schema names, table existence, and case sensitivity.

What We Chose and Why

We manually fixed the SQL scripts and database structure after understanding the root cause of each error. AI was used only for explanation, not for writing final queries.

Prompt 4: C# Class Structure and Responsibility

What We Asked

We asked AI whether my C# class structure (Entities, Services, Repositories, Program.cs) followed good design principles for a console-based system.

What AI Suggested

AI confirmed that the structure aligns with separation of concerns and suggested keeping Program.cs minimal while moving logic into services.

What I Chose and Why

We kept full control over the class design and logic flow. AI feedback was used only as validation, not as a decision-maker.

Prompt 5: Connecting C# Application to SQL Database

What We Asked

We asked AI conceptually whether it was worth connecting the C# project to the SQL database and what impact it would have on complexity and evaluation.

What AI Suggested

AI explained that database integration increases realism and project value but is not mandatory if both projects are complete and functional independently.

What We Chose and Why

We decided to treat integration as an optional enhancement (bonus), prioritizing stability and correctness of each project first.

Prompt 6: Debugging Database Connection Issues

What We Asked

We asked AI to explain common database connection issues in C# (connection strings, missing references, incorrect namespaces).

What AI Suggested

AI explained how database drivers work, how connection factories are structured, and how to interpret runtime exceptions.

What We Chose and Why

We fixed the issues manually after understanding the explanation. No connection logic was blindly copied.

Prompt 7: Code Quality and Maintainability Review

What We Asked

We asked AI to review my approach from a professional and academic perspective and to highlight potential weaknesses.

What AI Suggested

AI suggested focusing on clarity, consistent naming, meaningful console prompts, and avoiding overengineering.

What We Chose and Why

We applied only the recommendations that improved readability without changing core logic, ensuring the project remained our own work.

Prompt 8: Final Academic Evaluation Perspective

What We Asked

We asked AI to evaluate how the project would likely be graded with and without integration between C# and SQL.

What AI Suggested

AI explained expected academic evaluation criteria and emphasized that functionality, understanding, and documentation matter more than complexity.

What I Chose and Why

We focused on delivering a stable, understandable, and well-documented solution rather than risking late-stage architectural changes.

Final Statement

AI was used responsibly as a **learning assistant**, not as a solution generator. Every decision in this project was made consciously, with full understanding of the implemented logic.

This document ensures full transparency and compliance with the AI Usage Policy and can be safely included in the project repository.

****Student Declaration:**

We confirm that this project was developed collaboratively by two students. Both contributors fully understand every part of the system and can explain the implementation line-by-line if required.

AI was used strictly as a supportive learning tool, and all final decisions, implementations, and validations were made jointly by the project authors.